

Dyanamic Array

```
#include<iostream>

using namespace std;

class Array
{
    struct node{
        int *ptr;
        int lastIndex;
        int capacity;
    };
    node *arr;

public:
    Array(int cap){
        arr = new node;
        arr->capacity = cap;
        arr->lastIndex = -1;
        arr->ptr = new int[arr->capacity];
    }

    void DoubleArray(){
```

```
int *ptr2 = new int[arr->capacity*2];
for(int i=0 ; i<=arr->lastIndex ; i++)
    ptr2[i] = arr->ptr[i];
arr->ptr = ptr2;
arr->capacity *= 2;
}
```

```
void HalfArray(){
    int *ptr2 = new int[arr->capacity/2];
    for(int i=0 ; i<=arr->lastIndex ; i++)
        ptr2[i] = arr->ptr[i];
    arr->ptr = ptr2;
    arr->capacity /= 2;
}
```

```
void Append(int value){
    if(arr->lastIndex == arr->capacity-1)
        DoubleArray();
    arr->lastIndex += 1;
    arr->ptr[arr->lastIndex] = value;
}
```

```
int Search(int value){
    for(int i=0 ; i<=arr->lastIndex ; i++)
```

```
        if(arr->ptr[i] == value)
            return i;
    return -1;
}
```

```
void Insert(int index, int value){
    if(index<0 || index>arr->lastIndex+1)
        cout<<"Invalid Index";
    else if(index == arr->lastIndex+1)
        Append(value);
    else{
        if(arr->lastIndex == arr->capacity-1)
            DoubleArray();
        arr->lastIndex += 1;
        for(int i=arr->lastIndex ; i>index ; i--)
            arr->ptr[i] = arr->ptr[i-1];
        arr->ptr[index] = value;
    }
}
```

```
void Edit(int value, int newValue){
    int index = Search(value);
    if(index == -1)
        cout<<"%d not found in Array "<<value;
```

```
    else
        arr->ptr[index] = newValue;
}

void Delete(int value){
    int index = Search(value);
    if(index == -1)
        cout<<"%d not found in Array "<<value;
    else{
        for(int i=index ; i<arr->lastIndex ; i++)
            arr->ptr[i] = arr->ptr[i+1];
        arr->lastIndex -= 1;
        if(arr->lastIndex+2 < arr->capacity/2)
            HalfArray();
    }
}
```

```
void View(){
    if(arr->lastIndex == -1)
        cout<<"Array is Empty";
    else
        for(int i=0;i<=arr->lastIndex;i++)
            cout<<arr->ptr[i]<<" ";
}
```

```
};
```

```
int main()
```

```
{
```

```
    Array a1(5);
```

```
    a1.Append(10);
```

```
    a1.Append(20);
```

```
    a1.Append(40);
```

```
    a1.Append(50);
```

```
    a1.Insert(2,30);
```

```
    a1.Delete(50);
```

```
    a1.Edit(10,100);
```

```
    a1.View();
```

```
}
```

Linked List

```
#include<iostream>
```

```
using namespace std;
```

```
class List
```

```
{
```

```
    struct Node{
```

```
    int item;
    Node *next;
};
Node *start;
Node* Search(int value)
{
    Node *temp = start;
    for(int i; temp!=NULL ; i++){
        if(temp->item == value)
            return temp;
        temp = temp->next;
    }
    return NULL;
}

public:
    List(){
        start = NULL;
    }

    void View();
    void Append(int);
    void InsertFirst(int);
```

```
void InsertAfter(int, int);  
void DeleteFirst();  
void DeleteLast();  
void DeleteNode(int);  
  
};
```

```
void List::View()  
{  
    Node *temp = start;  
    while(temp != NULL){  
        cout<<temp->item<<" ";  
        temp = temp->next;  
    }  
}
```

```
void List::InsertFirst(int value)  
{  
    Node *n = new Node;  
    n->item = value;  
    n->next = start;  
    start = n;
```

```
}
```

```
void List::Append(int value)
```

```
{
```

```
    Node *n = new Node;
```

```
    n->item = value;
```

```
    n->next = NULL;
```

```
    Node *temp = start;
```

```
    if(start == NULL)
```

```
        start = n;
```

```
    else{
```

```
        while(temp->next != NULL)
```

```
            temp = temp->next;
```

```
        temp->next = n;
```

```
    }
```

```
}
```

```
void List::DeleteFirst()
```

```
{
```

```
    if(start == NULL)
```

```
        cout<<"UnderFlow";
```

```
    else{
```

```
        struct Node *temp = start;
```

```
        start = temp->next;
```



```

        delete temp;
    }

}

void List::DeleteLast()
{
    if(start == NULL)
        cout<<"UnderFlow";
    else if(start->next == NULL)
        DeleteFirst();
    else{
        Node *temp = start, *ttemp = NULL;
        while(temp->next != NULL){
            ttemp = temp;
            temp = temp->next;
        }
        ttemp->next = NULL;
        delete temp;
    }

}

void List::DeleteNode(int value)

```

```

{
    Node *temp = start, *ttemp = NULL;
    if(temp == NULL)
        cout<<value<<" not Found in List";
    else{
        while(temp->item != value){
            ttemp = temp;
            temp = temp->next;
        }
        if(ttemp == NULL)
            DeleteFirst();
        else{
            ttemp->next = temp->next;
            delete temp;
        }
    }
}

```

```

void List::InsertAfter(int value, int newValue)

```

```

{
    Node *temp = Search(value);
    if(temp == NULL)
        cout<<value<<" doesn't Exist";
    else{

```

```
    Node *n = new Node;
    n->item = newValue;
    n->next = temp->next;
    temp->next = n;
}
}
```

```
int main()
{
    List l1;
    l1.Append(10);
    l1.Append(20);
    l1.Append(30);
    l1.InsertFirst(5);
    l1.InsertAfter(30, 40);
    l1.InsertAfter(20, 25);
    l1.View();
    cout<<endl<<"After deleting 1st ";
    l1.DeleteFirst();
    l1.DeleteLast();
    l1.DeleteNode(20);
    l1.View();
}
```

Doubly Linked List

```
#include<iostream>

using namespace std;

class List
{
    struct Node{
        Node *prev;
        int item;
        Node *next;
    };
    Node *start;
    Node* Search(int value)
    {
        Node *temp = start;
        for(int i; temp!=NULL ; i++){
            if(temp->item == value)
                return temp;
            temp = temp->next;
        }
        return NULL;
    }
}
```

```
public:
```

```
List()
```

```
{
```

```
    start = NULL;
```

```
}
```

```
void View();
```

```
void InsertFirst(int);
```

```
void Append(int);
```

```
void InsertAfter(int, int);
```

```
};
```

```
void List::View()
```

```
{
```

```
    Node *temp = start;
```

```
    while(temp != NULL){
```

```
        cout<<temp->item<<" ";
```

```
        temp = temp->next;
```

```
    }
```

```
}
```

```
void List::InsertFirst(int value)
```

```
{
```

```
    Node *n = new Node;
```

```
n->prev = NULL;
n->item = value;
n->next = start;
if(start != NULL)
    start->prev = n;
start = n;
}
```

```
void List::InsertAfter(int value, int newValue)
{
    Node *n = new Node, *temp = Search(value);
    n->item = newValue;
    if(temp == NULL)
        cout<<value<<" Not Exist.";
    else{
        n->prev = temp;
        n->next = temp->next;
        if(temp->next != NULL)
            temp->next->prev = n;
        temp->next = n;
    }
}
```

```
void List::Append(int value)
```

```

{
    Node *n = new Node, *temp = start;
    n->item = value;
    n->next = NULL;
    if(start == NULL)
        InsertFirst(value);
    else{
        while(temp->next != NULL)
            temp = temp->next;
        n->prev = temp;
        temp->next = n;
    }
}

int main()
{
    List l1;
    l1.InsertFirst(20);
    l1.InsertFirst(10);
    l1.Append(30);
    l1.InsertAfter(50,15);
    l1.View();
}

```

Circular Linked List

```
#include<iostream>

using namespace std;

class List
{
    struct Node{
        int item;
        Node *next;
    };
    Node *last;
    Node* Search(int value)
    {
        if(last != NULL){
            struct Node *temp = last;
            while(temp->next != last){
                if(temp->item == value)
                    return temp;
                temp = temp->next;
            }
            if(temp->item == value)
                return temp;
        }
    }
}
```



```

        }
        return NULL;
    }
public:
    List()
    {
        last = NULL;
    }

    void View();
    void InsertFirst(int);
    void Append(int);
    void InsertAfter(int, int);
    void DeleteFirst();
    void DeleteLast();
    void DeleteNode(int);
};

void List::View()
{
    if(last == NULL)
        cout<<"Empty List";
    else{
        Node *temp = last->next;

```

```
        while(temp != last){  
            cout<<temp->item<<" ";  
            temp = temp->next;  
        }  
        cout<<temp->item<<endl;  
    }  
}  
void List::InsertFirst( int value)  
{  
    Node *n = new Node;  
    n->item = value;  
    if(last == NULL){  
        last = n;  
        n->next = n;  
    }  
    else{  
        n->next = last->next;  
        last->next = n;  
    }  
}
```

```
void List::Append(int value)  
{  
    Node *n = new Node;
```

```
n->item = value;
if(last == NULL)
    InsertFirst(value);
else{
    Node *temp = last;
    n->next = last-> next;
    last->next = n;
    last = n;
}
}
```

```
void List::InsertAfter(int value, int newValue)
{
    Node *temp = Search(value);
    if(temp == NULL)
        cout<<value<<" Not Found";
    else{
        Node *n = new Node;
        n->item = newValue;
        n->next = temp->next;
        temp->next = n;
        if(temp == last)
            last =n;
    }
}
```

```
}  
void List::DeleteFirst()  
{  
    if(last == NULL)  
        cout<<"UnderFlow";  
    else{  
        Node *temp = last->next;  
        if(last == temp)  
            last = NULL;  
        else  
            last->next = temp->next;  
        delete temp;  
    }  
}
```

```
void List::DeleteLast()  
{  
    if(last == NULL)  
        cout<<"UnderFlow";  
    else{  
        Node *temp = last->next, *ttemp = NULL;  
        if(last == temp)  
            DeleteFirst();  
        else{
```

```
        while(temp != last){  
            ttemp = temp;  
            temp = temp->next;  
        }  
        ttemp->next = temp->next;  
        last = ttemp;  
        delete temp;  
    }  
  
}
```

```
void List::DeleteNode(int value)  
{  
    Node *temp = Search(value);  
    if(temp == NULL)  
        cout<<value<<" Not Found";  
    else if(temp == last)  
        DeleteLast();  
    else{  
        Node *ttemp = last->next;  
        if(ttemp == last)  
            DeleteFirst();  
        else{
```

```
        while(ttemp->next != temp)
            ttemp = ttemp->next;
        ttemp->next = temp->next;
        delete temp;
    }
}

int main()
{
    List l1;
    l1.InsertFirst(10);
    l1.Append(20);
    l1.Append(30);
    l1.InsertAfter(20,25);
    // l1.DeleteFirst();
    //l1.DeleteLast();
    l1.View();
    l1.DeleteNode(20);
    l1.View();
}
```

Doubly Circular Linked List

```
#include<iostream>

using namespace std;

class List
{
    struct Node
    {
        int item;

        Node *prev,*next;
    };

    Node *start;

    Node* Search(int value)
    {
        Node *temp = start;

        if(temp != NULL)
            do{
                if(temp->item == value)
                    return temp;

                temp = temp->next;
            }while(temp != start);

        return NULL;
    }
}
```

public:

List()

{

start = NULL;

}

void View();

void InsertFirst(int);

void Append(int);

void InsertAfter(int, int);

void DeleteFirst();

void DeleteLast();

void DeleteNode(int);

};

void List::View()

{

Node *temp = start;

if(start == NULL)

cout<<"Empty.";

else

do{

cout<<temp->item<<" ";

temp = temp->next;

}while(temp != start);


```
        cout<<endl;
    }

void List::InsertFirst(int value)
{
    Node *n = new Node;
    n->item = value;
    if(start == NULL){
        n->next = n;
        n->prev = n;
    }
    else{
        n->next = start;
        n->prev = start->prev;
        start->prev->next = n;
        start->prev = n;
    }
    start = n;
}
```

```
void List::Append(int value)
{
    Node *n = new Node;
    n->item = value;
```

```
if(start == NULL)
    InsertFirst(value);
else{
    n->next = start;
    n->prev = start->prev;
    start->prev->next = n;
    start->prev = n;
}
}

void List::InsertAfter(int value, int newValue)
{
    Node *temp = Search(value);
    if(temp == NULL)
        cout<<value<<" not found";
    else{
        Node *n = new Node;
        n->item = newValue;
        n->next = temp->next;
        n->prev = temp;
        temp->next->prev = n;
        temp->next = n;
    }
}
```

```
void List::DeleteFirst()
{
    if(start == NULL)
        cout<<"UnderFlow";
    else{
        Node *temp = start;
        if(temp->next == start)
            start = NULL;
        else{
            start->prev->next = start->next;
            start->next->prev = start->prev;
            start = start->next;
        }
        delete temp;
    }
}
```

```
void List::DeleteLast()
{
    if(start == NULL)
        cout<<"UnderFlow."<<endl;
    else{
        Node *temp = start->prev;
```

```

        if(temp == start)
            start = NULL;
        else{
            temp->prev->next = start;
            start->prev = temp->prev;
        }
        delete temp;
    }

}

void List::DeleteNode(int value)
{
    Node *temp = Search(value);
    if(temp == NULL)
        cout<<value<<" Not Found"<<endl;
    else{
        if(temp == start)
            DeleteFirst();
        else{
            temp->prev->next = temp->next;
            temp->next->prev = temp->prev;
            delete temp;
        }
    }
}

```

```
    }  
}  
  
int main()  
{  
    List l1;  
    l1.InsertFirst(20);  
    l1.InsertFirst(10);  
    l1.Append(30);  
    l1.Append(40);  
    l1.InsertFirst(5);  
    l1.InsertAfter(5,7);  
    l1.InsertAfter(30,35);  
    l1.InsertAfter(40,45);  
    l1.View();  
    l1.DeleteFirst();  
    l1.DeleteFirst();  
    l1.DeleteFirst();  
    l1.View();  
    l1.DeleteLast();  
    l1.View();  
    l1.DeleteNode(35);  
    l1.View();  
}
```

Stack with Linked List

```
#include<iostream>

using namespace std;
```

```
class Stack
{
    struct Node{
        int item;
        Node *next;
    };
    Node *start;
public:
    List(){
        start = NULL;
    }

    void PEEK();
    void PUSH(int);
    void POP();
};

int List::PEEK()
```

```
{  
    Node *temp =start;  
    if(temp != NULL){  
        return temp->item;  
    }  
    else{  
        cout<<"Empty";  
    }  
}
```

```
void List::PUSH(int value)
```

```
{  
    Node *n = new Node;  
    n->item = value;  
    n->next = start;  
    start = n;  
}
```

```
void List::POP()
```

```
{  
    if(start == NULL)  
        cout<<"UnderFlow";  
    else{  
        struct Node *temp = start;
```

```
        start = temp->next;
        delete temp;
    }
}
```

Reverse a Linked List

```
#include<iostream>
using namespace std;
class List
{
    struct Node{
        int item;
        Node *next;
    };
    Node *start;
    Node* Search(int value)
    {
        Node *temp = start;
        for(int i; temp!=NULL ; i++){
            if(temp->item == value)
                return temp;
            temp = temp->next;
        }
    }
}
```



```
}
```

```
return NULL;
```

```
}
```

```
public:
```

```
List(){
```

```
    start = NULL;
```

```
}
```

```
void View();
```

```
void Append(int);
```

```
void InsertFirst(int);
```

```
void InsertAfter(int, int);
```

```
void DeleteFirst();
```

```
void DeleteLast();
```

```
void DeleteNode(int);
```

```
void ReverseList();
```

```
void ReverseList1();
```

```
};
```

```
void List::View()
{
    Node *temp = start;
    while(temp != NULL){
        cout<<temp->item<<" ";
        temp = temp->next;
    }
}
```

```
void List::InsertFirst(int value)
{
    Node *n = new Node;
    n->item = value;
    n->next = start;
    start = n;
}
```

```
void List::Append(int value)
{
    Node *n = new Node;
    n->item = value;
    n->next = NULL;
    Node *temp = start;
    if(start == NULL)
```

```
        start = n;
    else{
        while(temp->next != NULL)
            temp = temp->next;
        temp->next = n;
    }
}
```

```
void List::DeleteFirst()
{
    if(start == NULL)
        cout<<"UnderFlow";
    else{
        struct Node *temp = start;
        start = temp->next;
        delete temp;
    }
}
```

```
void List::DeleteLast()
{
    if(start == NULL)
        cout<<"UnderFlow";
```

```
else if(start->next == NULL)
    DeleteFirst();
else{
    Node *temp = start, *ttemp = NULL;
    while(temp->next != NULL){
        ttemp = temp;
        temp = temp->next;
    }
    ttemp->next = NULL;
    delete temp;
}

}
```

```
void List::DeleteNode(int value)
{
    Node *temp = start, *ttemp = NULL;
    if(temp == NULL)
        cout<<value<<" not Found in List";
    else{
        while(temp->item != value){
            ttemp = temp;
            temp = temp->next;
        }
    }
```

```
    if(ttemp == NULL)
        DeleteFirst();
    else{
        ttemp->next = temp->next;
        delete temp;
    }
}
```

```
void List::InsertAfter(int value, int newValue)
{
    Node *temp = Search(value);
    if(temp == NULL)
        cout<<value<<" doesn't Exist";
    else{
        Node *n = new Node;
        n->item = newValue;
        n->next = temp->next;
        temp->next = n;
    }
}
```

```
void List::ReverseList()
{
```

```
if(start->next != NULL){  
    Node *temp = start->next;  
    Node *ttemp = temp->next;  
  
    start->next = NULL;  
    while(temp != NULL ){  
        temp->next = start;  
        start = temp;  
        temp = ttemp;  
        if(ttemp != NULL)  
            ttemp = ttemp->next;  
    }  
}  
else  
    cout<<"One Value";  
}
```

```
void List::ReverseList1()  
{  
  
    if(start->next != NULL){  
        Node *temp = start->next;  
        Node *ttemp = temp->next;  
    }  
}
```

```
        else
            cout<<"One Value";
    }
int main()
{
    List l1;
    l1.Append(10);
    l1.Append(20);

    l1.View();
    l1.ReverseList();

    cout<<endl;
    l1.View();
}
```

Check cycle in Linked List

```
#include<iostream>
#include<stdio.h>
using namespace std;
class List
```

```
{
public:
    int Data;
    List *Next;
};
int CycleInLinkedlist(List *temp)
{

    if(temp != NULL){

        List *ttemp = temp->Next;

        while( temp != ttemp ){
            if(temp == NULL || ttemp->Next == NULL || ttemp == NULL)
                return 0;
            temp = temp->Next;
            ttemp = ttemp->Next->Next;
        }

        int Node = 0;
        do{
            Node++;
            temp = temp->Next;
        }while( temp != ttemp );
```



```
        return Node;
    }
}

int main()
{
    List *First=new List;
    First->Data=10;
    First->Next=NULL;

    List *Sec=new List;
    First->Next=Sec;
    Sec->Data=20;
    Sec->Next=NULL;

    List *Third=new List;
    Sec->Next=Third;
    Third->Data=30;
    Third->Next=NULL;

    List *Fourth=new List;
    Third->Next=Fourth;
    Fourth->Data=40;
    Fourth->Next= Fourth;
}
```

```
List *start = First;

cout<<"Number of Node in cycle in the Linked List is "<<CycleInLinkedlist(start);

}
```

TOH

```
#include<iostream>

using namespace std;

void TOH(char main, char alt, char dest, int ring)
{
    if(ring>0){
        TOH(main, dest, alt, ring-1);
        cout<<main<<"--"<<dest<<" ";
        TOH(alt, main, dest, ring-1);
    }
}

int main()
```

```
{  
    int ring = 4;  
    TOH('A', 'B', 'C', ring);  
}
```

TWO WAY STACK

```
#include<iostream>  
  
using namespace std;  
  
class STACK  
{  
    struct TwoWayStack  
    {  
        int capacity;  
        int *ptr;  
        int top1, top2;  
    };  
    TwoWayStack *sta;  
public:  
    STACK(int cap)  
    {  
        sta = new TwoWayStack;
```

```
    sta->capacity = cap;
    sta->ptr = new int[cap];
    sta->top1 = -1;
    sta->top2 = cap;
}
```

```
void PUSH_STACK1(int);
int PEEK_STACK1();
int POP_STACK1();
void PUSH_STACK2(int);
int PEEK_STACK2();
int POP_STACK2();

};
```

```
void STACK::PUSH_STACK1(int value)
{
    if(sta->top1+1 == sta->top2)
        cout<<"OverFlow"<<endl;
    else{
        sta->top1++;
        sta->ptr[sta->top1] = value;
```

```
    }  
}  
  
void STACK::PUSH_STACK2(int value)
```

```
{  
    if(sta->top2-1 == sta->top1)  
        cout<<"OverFlow"<<endl;  
    else{  
        sta->top2--;  
        sta->ptr[sta->top2] = value;  
    }  
}
```

```
int STACK::POP_STACK1()  
{  
    if(sta->top1 == -1)  
        cout<<"UnderFlow"<<endl;  
    else{  
        sta->top1--;  
        return sta->ptr[sta->top1+1];  
    }  
}
```

```
int STACK::POP_STACK2()
```

```
{  
    if(sta->top2 == sta->capacity)  
        cout<<"UnderFlow"<<endl;  
    else{  
        sta->top2++;  
        return sta->ptr[sta->top2-1];  
    }  
}
```

```
int STACK::PEEK_STACK1()  
{  
    if(sta->top1 == -1)  
        cout<<"UnderFlow"<<endl;  
    else  
        return sta->ptr[sta->top1];  
}
```

```
int STACK::PEEK_STACK2()  
{  
    if(sta->top2 == sta->capacity)  
        cout<<"UnderFlow"<<endl;  
    else  
        return sta->ptr[sta->top2];  
}
```

```
}
```

```
int main()
```

```
{
```

```
    STACK s1(10);
```

```
    s1.PUSH_STACK1(10);
```

```
    s1.PUSH_STACK1(20);
```

```
    s1.PUSH_STACK1(30);
```

```
    s1.PUSH_STACK1(40);
```

```
    s1.PUSH_STACK1(50);
```

```
    s1.PUSH_STACK2(100);
```

```
    s1.PUSH_STACK2(200);
```

```
    s1.PUSH_STACK2(300);
```

```
    s1.PUSH_STACK2(400);
```

```
    s1.PUSH_STACK2(500);
```

```
    s1.POP_STACK1();
```

```
    s1.PUSH_STACK2(600);
```

```
    cout<<s1.PEEK_STACK1()<<endl;
```

```
    cout<<s1.PEEK_STACK2();
```

```
}
```

Circular Queue by Dynamic Array

```
#include<iostream>

#include<conio.h>

using namespace std;

class Queue
{
    struct QueueInfo
    {
        int first, last, capacity, *ptr;
    };
    QueueInfo *start;
    void IncreaseArray()
    {
        int *temp = new int[start->capacity+5];
        if(start->last != -1){
            int num = -1;
            do{
                num++;
                if(start->last == start->capacity)
                    start->last = 0;
                *(temp+num) = *(start->ptr+start->last);
                start->last++;
            } while (start->last != -1);
            delete[] start->ptr;
            start->ptr = temp;
            start->capacity = start->capacity+5;
        }
    }
};
```



```

        }while(start->last-1 != start->first);
        start->last = 0;
        start->first = num;
        delete start->ptr;
    }
    start->capacity += 5;
    start->ptr = temp;
}

void DecreaseArray()
{
    if(start->last == -1){
        delete start->ptr;
        start->ptr = NULL;
        start->capacity = 0;
    }
    else{
        int *temp = new int[start->capacity-5], num = -1;
        do{
            num++;
            if(start->last == start->capacity)
                start->last = 0;
            *(temp+num) = *(start->ptr+start->last);
            start->last++;
        }while(start->last-1 != start->first);
    }
}

```

```

        start->last = 0;
        start->first = num;
        start->capacity = start->capacity-5;
        delete start->ptr;
        start->ptr = temp;
    }
}
public:
    Queue()
    {
        start = new QueueInfo;
        start->last = -1;
        start->first = -1;
        start->capacity = 0;
        start->ptr = NULL;
    }
    void Insert(int);
    int Delete();
    void ViewAll();
};
void Queue::Insert(int data)
{
    if(start->first-start->last == start->capacity-1 || start->last-start->first == 1 ||
start->last == -1){
        //cout<<"pass";

```

```

        //getch();
        IncreaseArray();
    }
    if(start->first == -1){
        *(start->ptr+(start->first+1)) = data;
        start->first++;
        start->last++;
    }
    else{
        start->first++;
        //cout<<"test"<<endl;
        if(start->first==start->capacity)
            start->first = 0;
        *(start->ptr+start->first) = data;
    }
}

int Queue::Delete()
{
    if(start->last == -1){
        cout<<"Queue Underflow"<<endl;
        return NULL;
    }
    else if(start->first == start->last){
        int temp = *(start->ptr+(start->last));

```

```

        start->last = -1;
        start->first = -1;
        DecreaseArray();
        return temp;
    }
    else{
        int temp = *(start->ptr+start->last);
        if(start->last == start->capacity-1)
            start->last = -1;
        start->last++;
        if(start->first-start->last < start->capacity-5 || start->last-start->first > start-
>capacity-5)
            DecreaseArray();
        return temp;
    }
}

void Queue::ViewAll()
{
    if(start->last == -1)
        cout<<"Queue Underflow!";
    else{
        int num = start->last;
        do{
            cout<<*(start->ptr+num)<<" ";
            num++;
        }
    }
}

```

```

        if(num==start->capacity)
            num = 0;
        }while(num-1 != start->first);
    }
    cout<<endl;
    cout<<"cap - "<<start->capacity<<endl<<"last - "<<start->last<<endl<<"first - 
"<<start->first<<endl;
    getch();
}

int main()
{
    Queue q;
    q.Insert(0);
    q.Insert(1);
    q.Insert(2);
    q.Insert(3);
    q.Insert(4);
    cout<<q.Delete()<<endl;
    cout<<q.Delete()<<endl;
    q.Insert(6);
    q.Insert(7);
    cout<<q.Delete()<<endl;
    cout<<q.Delete()<<endl;
    cout<<q.Delete()<<endl;
}

```

DEQUEUE

```
#include<iostream>

using namespace std;

class DEQUEUE
{
    struct Queue{
        int Head, Tail;
        int *ptr;
        int capacity;
    };
    Queue *deq;

public:
    DEQUEUE(int cap)
    {
        deq = new Queue;
        deq->Head = deq->Tail = -1;
        deq->ptr = new int[cap];
        deq->capacity = cap;
    }
```

```

void InsertAsHead(int);
void InsertAsTail(int);
void DeleteHead();
void DeleteTail();
void View();
};

void DEQUEUE::InsertAsHead(int value)
{
    if(deq->Head==0 && deq->Tail==deq->capacity-1 || deq->Tail+1 == deq->Head
)
        cout<<"OverFlow"<<endl;
    else{
        if(deq->Head == -1)
            deq->Head = deq->Tail = 0;
        else if(deq->Head == 0)
            deq->Head = deq->capacity-1;
        else
            deq->Head--;
        deq->ptr[deq->Head] = value;
    }
}

void DEQUEUE::InsertAsTail(int value)

```

```

{
    if(deq->Head==0 && deq->Tail==deq->capacity-1 || deq->Tail+1 == deq->Head
)
        cout<<"OverFlow"<<endl;
    else{
        if(deq->Tail == -1)
            deq->Head = deq->Tail = 0;
        else if(deq->Tail == deq->capacity-1)
            deq->Tail = 0;
        else
            deq->Tail++;
        deq->ptr[deq->Tail] = value;
    }
}

```

```

void DEQUEUE::DeleteHead()
{
    if(deq->Head == -1)
        cout<<"UnderFlow"<<endl;
    if(deq->Head == deq->Tail)
        deq->Head = deq->Tail = -1;
    else if(deq->Head == deq->capacity-1)
        deq->Head = 0;
    else
        deq->Head++;
}

```



```
}
```

```
void DEQUEUE::DeleteTail()
```

```
{
```

```
    if(deq->Tail == -1)
```

```
        cout<<"UnderFlow"<<endl;
```

```
    if(deq->Head == deq->Tail)
```

```
        deq->Head = deq->Tail = -1;
```

```
    else if(deq->Tail == 0)
```

```
        deq->Tail= deq->capacity-1;
```

```
    else
```

```
        deq->Tail--;
```

```
}
```

```
void DEQUEUE::View()
```

```
{
```

```
    if(deq->Head == -1)
```

```
        cout<<"Empty"<<endl;
```

```
    else{
```

```
        int i;
```

```
        for (i=deq->Tail; deq->Head!=i ; i==0 ? i=deq->capacity-1 : i--)
```

```
            cout<<deq->ptr[i]<<" ";
```

```
        cout<<deq->ptr[i]<<endl;
```

```
    }
```

```
}
```

```
int main()
```

```
{
```

```
    DEQUEUE d1(10);
```

```
    d1.InsertAsHead(10);
```

```
    d1.InsertAsHead(20);
```

```
    d1.InsertAsHead(30);
```

```
    d1.InsertAsHead(40);
```

```
    d1.InsertAsTail(200);
```

```
    d1.InsertAsTail(300);
```

```
    d1.InsertAsTail(400);
```

```
    d1.InsertAsTail(100);
```

```
    d1.DeleteTail();
```

```
    d1.DeleteTail();
```

```
    d1.DeleteTail();
```

```
    d1.DeleteTail();
```

```
    d1.DeleteTail();
```

```
    d1.View();
```

```
}
```

PRIORITY QUEUE

```
#include<iostream>

using namespace std;

class PQ
{
    struct PriorityQ
    {
        int *Front, *Rare;
        int **ptr;
        int prio,cap;
    };
    PriorityQ *pque;
public:
    PQ(int cap,int prio)
    {
        pque = new PriorityQ;
        pque->ptr = new int*[prio];
        pque->Front = new int[prio];
        pque->Rare = new int[prio];
        for(int i=0 ; i<prio; i++){
            pque->ptr[i] = new int[cap];
            pque->Front[i] = pque->Rare[i] = -1;
        }
    }
};
```

```
    }  
    pque->prio = prio;  
    pque->cap = cap;  
}
```

```
void View_Max_priority();  
void Delete_Max_priority();  
void Insertwith_priority(int, int);  
};
```

```
void PQ::View_Max_priority()  
{  
    int p;  
    for(p=0; pque->Front[p]==-1 && p<pque->prio ; p++);  
  
    if(p == pque->prio)  
        cout<<"UnderFlow p="<<p<<endl;  
    else  
        cout<<pque->ptr[p][pque->Rare[p]]<<endl;  
}
```

```
void PQ::Delete_Max_priority()  
{  
    int p;
```

```
for(p=0; pque->Front[p]==-1 && p<pque->prio ; p++);
```

```
if(p == pque->prio)
```

```
    cout<<"UnderFlow"<<endl;
```

```
else
```

```
    if(pque->Front[p] == pque->cap-1)
```

```
        pque->Front[p] = 0;
```

```
    else if(pque->Front[p] == pque->Rare[p]){
```

```
        pque->Front[p] = pque->Rare[p] = -1;
```

```
        cout<<"yes p="<<p<<endl;
```

```
    }
```

```
    else
```

```
        pque->Front[p]++;
```

```
}
```

```
void PQ::Insertwith_priority(int Data, int prio)
```

```
{
```

```
    if(pque->Front[prio]==0 && pque->Rare[prio]==pque->cap-1 ||
```

```
        pque->Rare[prio]+1 == pque->Front[prio])
```

```
        cout<<"Overloading with priority "<<prio<<endl;
```

```
    else{
```

```
        if(pque->Rare[prio] == -1)
```

```
            pque->Rare[prio] = pque->Front[prio] = 0;
```

```

else if(pque->Rare[prio] == pque->cap-1)
    pque->Rare[prio] = 0;
else{
    pque->Rare[prio]++;
    cout<<"yes ";
}
pque->ptr[prio][pque->Rare[prio]] = Data;
cout<<"Insert "<<Data<<" in "<<prio<<" priority front="<<pque-
>Front[prio]<<" rare="<<pque->Rare[prio]<<endl;
}

}

```

```

int main()
{
    PQ p1(5,4);
    p1.Insertwith_priority(10,0);
    p1.Insertwith_priority(20,0);
    p1.Insertwith_priority(100,0);
    p1.Insertwith_priority(200,0);
}

```

Binary Search Tree(insert, delete, traverse)

```
#include<iostream>

using namespace std;

class BST
{
    struct Node
    {
        Node *left;
        int item;
        Node *right;
    };
    void Preorder(Node*);
    void Inorder(Node*);
    void Postorder(Node*);
    void InsertRecursionChild(Node*, Node*);
    void DeleteZero(Node* ,Node*);
    void DeleteSingle(Node* ,Node*);
    void DeleteDouble(Node*);
    Node* DeleteRecursionChild(Node *temp, int data)
    {
        if(temp == NULL)
            return NULL;
```

```
if(temp->item > data)
    temp->left = DeleteRecursionChild(temp->left, data);
else if(temp->item < data)
    temp->right = DeleteRecursionChild(temp->right, data);
else if(temp->item == data){
    if(temp->left == NULL){
        Node *ttemp = temp->right;
        delete temp;
        return ttemp;
    }
    if(temp->right == NULL){
        Node *ttemp = temp->left;
        delete temp;
        return ttemp;
    }
    Node *ttemp = temp->left;
    while(ttemp->right)
        ttemp = ttemp->right;
    temp->item = ttemp->item;
    temp->left = DeleteRecursionChild(temp->left, ttemp->item);
}
return temp;
}
```


public:

Node *root ;

BST()

{

root = NULL;

}

void View();

void Insert(int);

void InsertRecursion(int);

void Delete(int);

void DeleteByRecursion(int);

};

void BST::Preorder(Node *temp)

{

if(temp){

cout<<temp->item<<" ";

Preorder(temp->left);

Preorder(temp->right);

}

}

void BST::Inorder(Node *temp)

```
{
    if(temp){
        Inorder(temp->left);
        cout<<temp->item<<" ";
        Inorder(temp->right);
    }
}

void BST::Postorder(Node *temp)
{
    if(temp){
        Postorder(temp->left);
        Postorder(temp->right);
        cout<<temp->item<<" ";
    }
}

void BST::View()
{
    Inorder(root);
    cout<<endl;
}

// Insertion code starts from here
```

```
void BST::Insert(int data)
{
    Node *n = new Node;
    n->left = n->right = NULL;
    n->item = data;
    if(root == NULL)
        root = n;
    else{
        Node *temp = root;
        while(1)
            if(temp->item > data)
                if(temp->left == NULL){
                    temp->left = n;
                    break;
                }
            else
                temp = temp->left;
        else
            if(temp->right == NULL){
                temp->right = n;
                break;
            }
        else
            temp = temp->right;
```

```
    }  
}
```

// Insertion with recursion

```
void BST::InsertRecursionChild(Node *temp, Node *n)
```

```
{  
    if(temp->item > n->item)  
        if(temp->left == NULL)  
            temp->left = n;  
        else  
            InsertRecursionChild(temp->left, n);  
    else  
        if(temp->right == NULL)  
            temp->right = n;  
        else  
            InsertRecursionChild(temp->right, n);  
}
```

```
void BST::InsertRecursion(int data)
```

```
{  
    Node *n = new Node;  
    n->left = NULL;  
    n->item = data;
```

```
n->right = NULL;
if(root == NULL)
    root = n;
else
    InsertRecursionChild(root, n);
}
```

// Deletion Code starts from here

```
void BST::DeleteZero(Node *ptr, Node *parPtr)
{
    if(parPtr)
        if(parPtr->left == ptr)
            parPtr->left = NULL;
        else
            parPtr->right = NULL;
    else
        root = NULL;
    delete ptr;
}
```

```
void BST::DeleteSingle(Node *ptr, Node *parPtr)
{

```

```

if(parPtr)
    if(parPtr->left == ptr)
        if(ptr->left)
            parPtr->left = ptr->left;
        else
            parPtr->left = ptr->right;
    else
        if(ptr->left)
            parPtr->right = ptr->left;
        else
            parPtr->right = ptr->right;
    else
        if(ptr->left)
            root = ptr->left;
        else
            root = ptr->right;

delete ptr;
}

void BST::DeleteDouble(Node *ptr)
{
    Node *temp = ptr->left, *parTemp = ptr;
    while(temp->right){

```

```

        parTemp = temp;
        temp = temp->right;
    }
    ptr->item = temp->item;
    if(temp->left)
        DeleteSingle(temp, parTemp);
    else
        DeleteZero(temp, parTemp);
}

void BST::Delete(int data)
{
    if(root == NULL)
        cout<<"UnderFlow"<<endl;
    else{
        Node *temp = root, *parTemp=NULL;
        while(temp->item != data && temp){
            parTemp = temp;
            if(temp->item > data)
                temp = temp->left;
            else
                temp = temp->right;
        }
        if(temp)

```

```

        if(temp->left == temp->right)
            DeleteZero(temp, parTemp);
        else if(temp->left==NULL || temp->right==NULL )
            DeleteSingle(temp, parTemp);
        else
            DeleteDouble(temp);
    else
        cout<<data<<" Not Found."<<endl;
    }
}

```

```

void BST::DeleteByRecursion(int data)
{
    root = DeleteRecursionChild(root, data);
}

int main()
{
    BST b1;
    b1.Insert(55);
    b1.Insert(40);
    b1.Insert(70);
    b1.Insert(30);
    b1.Insert(45);
}

```



```
b1.Insert(65);
b1.InsertRecursion(90);
b1.InsertRecursion(43);
b1.InsertRecursion(67);
b1.InsertRecursion(66);
b1.InsertRecursion(69);
b1.InsertRecursion(16);
b1.View();
b1.DeleteByRecursion(30);
b1.DeleteByRecursion(16);
b1.DeleteByRecursion(18);
//b1.DeleteByRecursion();
b1.View();
}
```

Bubble Sort

```
#include<iostream>
using namespace std;
void Bubble_sort(int *a, int Size)
{
    for(int i=1,done=0; i<Size; i++){
        for(int j=0; j<Size-i; j++)
```

```
        if(a[j]>a[j+1]){
            int temp = a[j+1];
            a[j+1] = a[j];
            a[j] = temp;
            cout<<"hello"<<endl;
            done++;
        }
        if(done==0)
            break;
    }
}

int main()
{
    //int a[10] = {50,80,40,20,100,60,30,70,90,10};
    int a[10] = {10,20,30,40,50,60,70,80,90,100};
    Bubble_sort(a,10);
    for(int i=0; i<10; i++)
        cout<<a[i]<<" ";
}
```

Selection Sort

```
#include<iostream>

using namespace std;

void Selection_Sort(int *a, int Size)
{
    int i,j,index,sm;
    for(i=0;i<Size;i++){
        sm=a[i];
        index = i;
        for(j=i+1;j<Size;j++)
            if(sm>a[j]){
                sm = a[j];
                index = j;
            }
        a[index] = a[i];
        a[i] = sm;
    }

}

int main()
{
    int a[10] = {50,80,40,20,100,60,30,70,90,10};
```

```
//int a[10] = {10,20,30,40,50,60,70,80,90,100};  
Selection_Sort(a,10);  
for(int i=0; i<10; i++)  
    cout<<a[i]<<" ";  
}
```

Insertion Sort

```
#include<iostream>  
using namespace std;  
void Insertion_Sort(int *a, int Size)  
{  
    int i,j,temp;  
    for(i=0; i<Size;i++)  
        for(j=i; j>=0; j--)  
            if(a[j]<a[j+1])  
                break;  
            else{  
                temp = a[j+1];  
                a[j+1] = a[j];  
                a[j] = temp;  
            }  
}
```

```
int main()
{
    int a[10] = {50,80,40,20,100,60,30,70,90,10};
    //int a[10] = {10,20,30,40,50,60,70,80,90,100};
    Insertion_Sort(a,10);
    for(int i=0; i<10; i++)
        cout<<a[i]<<" ";
}
```

File Sorting program

```
#include<iostream>
#include<string.h>
using namespace std;
struct Date{
    int day, month, year;
};
struct File{
    char fileName[20];
    Date date;
};
void sortFile(File *files, int Size)
```

```
{  
    File temp;  
    for(int i=1; i<Size; i++)  
        for(int j=0; j<Size-i; j++)  
            if(files[j].date.year > files[j+1].date.year){  
                temp = files[j];  
                files[j] = files[j+1];  
                files[j+1] = temp;  
            }  
        else  
            if(files[j].date.year == files[j+1].date.year)  
                if(files[j].date.month > files[j+1].date.month){  
                    temp = files[j];  
                    files[j] = files[j+1];  
                    files[j+1] = temp;  
                }  
            else  
                if(files[j].date.month == files[j+1].date.month)  
                    if(files[j].date.day > files[j+1].date.day){  
                        temp = files[j];  
                        files[j] = files[j+1];  
                        files[j+1] = temp;  
                    }  
            }  
}
```

```
void createFile(File *files)
{
    strcpy(files[0].fileName, "My Computer");
    files[0].date.day = 25;
    files[0].date.month = 12;
    files[0].date.year = 2021;

    strcpy(files[1].fileName, "My Programs");
    files[1].date.day = 21;
    files[1].date.month = 4;
    files[1].date.year = 2020;

    strcpy(files[2].fileName, "My Network");
    files[2].date.day = 29;
    files[2].date.month = 3;
    files[2].date.year = 2020;

    strcpy(files[3].fileName, "My WorkSpace");
    files[3].date.day = 30;
    files[3].date.month = 4;
    files[3].date.year = 2022;

    strcpy(files[4].fileName, "My Team");
    files[4].date.day = 30;
```

```
files[4].date.month = 4;
files[4].date.year = 2020;
}

void showData(File *files, int Size)
{
    for(int i=0; i<Size; i++)
        cout<<files[i].fileName<<" ---
"<<files[i].date.day<<"/"<<files[i].date.month<<"/"<<files[i].date.year<<endl;
}

int main()
{
    File files[5];
    createFile(files);
    showData(files, 5);
    sortFile(files,5);
    cout<<endl;
    showData(files, 5);
}
```


Quick Sort

```
#include<iostream>

using namespace std;

int quickSort(int *a, int low, int high)
{
    int left = low, right = high, pivot = a[low], temp;
    while(left < right){
        while(a[left] <= pivot )
            left++;
        while(a[right] > pivot)
            right--;
        if(left < right){
            temp = a[left];
            a[left] = a[right];
            a[right] = temp;
        }
    }
    temp = a[right];
    a[right] = a[low];
    a[low] = temp;
```

```

        return right;
    }

void Quick(int *a, int low, int high)
{
    if(low < high){
        int pivot = quickSort(a, low, high);
        Quick(a, low, pivot-1);
        Quick(a, pivot+1, high);
    }
}

int main()
{
    int a[5] = {88,22,44,55,11};
    Quick(a, 0, 4);
    for(int i = 0; i<5; i++)
        cout<<a[i]<<" ";

}

```

Merge Sort

```

#include<iostream>

using namespace std;

void merg(int *A, int low, int mid, int high)
{
    int h = high-low, m=mid-low, temp[high-low+1], i=0, j=mid-low+1;

    for(int t=0; t<=h ; t++)
        temp[t] = A[low+t];

    for(int k=low; k <= high; k++)
        if(i > m)
            A[k] = temp[j++];
        else if(j > h)
            A[k] = temp[i++];
        else if(temp[i] < temp[j])
            A[k] = temp[i++];
        else
            A[k] = temp[j++];
}

void mergSort(int *A, int low, int high)
{
    int mid = (low+high)/2;

```

```
if(low != high){  
    mergSort(A, low, mid);  
    mergSort(A, mid+1, high);  
    merg(A, low, mid, high);  
}  
}
```

```
int main()  
{  
    int a[11] = {88,22,44,55,11,33,66,99,77,111,5};  
    mergSort(a, 0, 10);  
    for(int i = 0; i<11; i++)  
        cout<<a[i]<<" ";  
  
}
```

Heap Sort

```
#include<iostream>  
using namespace std;  
int heap[10];
```

```
class Heap
{
    struct heapArr
    {
        int lastIndex, capacity;
        int *H;
    };
    heapArr *node;
public:
    Heap(int cap)
    {
        node = new heapArr;
        node->lastIndex = -1;
        node->capacity = cap;
        node->H = new int[cap];
    }
    void insertHeap(int);
    void deletHeap();
    void viewHeap();
    void viewHeapSort();
};

void Heap::viewHeapSort()
{
```

```
for(int i=0; i<10; i++)
    cout<<node->H[i]<<" ";
cout<<endl;
}
void Heap::viewHeap()
{
    for(int i=0; i<=node->lastIndex; i++)
        cout<<node->H[i]<<" ";
    cout<<endl;
}
void Heap::insertHeap(int data)
{
    if(node->lastIndex == node->capacity-1)
        cout<<"Overflow";
    else{
        int index = ++node->lastIndex;
        while(index)
            if(node->H[(index-1)/2] < data){
                node->H[index] = node->H[(index-1)/2];
                index = (index-1)/2;
            }
            else
                break;
        node->H[index] = data;
    }
}
```

```

    }
}

void Heap::deletHeap()
{
    if(node->lastIndex == -1)
        cout<<"UnderFlow";
    else{
        int temp = node->H[node->lastIndex], index = 0;
        node->H[node->lastIndex] = node->H[0];
        node->lastIndex--;
        while(index*2 < node->lastIndex)
            if(node->H[index*2+1] > node->H[index*2+2] || index*2+1 == node->lastIndex)
                if(temp < node->H[index*2+1]){
                    node->H[index] = node->H[index*2+1];
                    index = index*2+1;
                }
            else
                break;
        else
            if(temp < node->H[index*2+2]){
                node->H[index] = node->H[index*2+2];
                index = index*2+2;
            }
    }
}

```

```
        else
            break;
        node->H[index] = temp;
    }
}

int main()
{
    Heap h1(10);
    h1.insertHeap(30);
    h1.insertHeap(40);
    h1.insertHeap(60);
    h1.insertHeap(70);
    h1.insertHeap(80);
    h1.insertHeap(10);
    h1.insertHeap(11);
    h1.insertHeap(20);
    h1.insertHeap(25);
    h1.insertHeap(90);
    h1.deletHeap();
    h1.deletHeap();
    // h1.deletHeap();
    // h1.deletHeap();
    // h1.deletHeap();
    // h1.deletHeap();
}
```



```
// h1.deletHeap();  
    h1.viewHeapSort();  
}
```

Binary Search

```
#include<iostream>  
  
using namespace std;  
  
int binarySearch(int *A, int low, int high, int data)  
{  
    int mid = (low+high)/2;  
  
    if(A[mid] == data)  
        return mid;  
    if(mid == high || mid == low-1)  
        return -1;  
  
    if(data > A[mid])  
        return binarySearch(A, mid+1, high, data);  
    return binarySearch(A, low, mid, data);  
}  
  
int main()
```

```

{
    int A[10] = {5,11,22,33,44,55,66,77,88,99};
    for(int i = 0; i<10; i++)
        cout<<A[i]<<" ";
    cout<<endl<<"44 found at "<<binarySearch(A, 0, 9, 99)<<" Index.";
}

```

Hashing

```

#include<iostream>
#include<string.h>
using namespace std;

class Student
{
    int row;
    struct student{
        int rollNum;
        char name[25];
        struct student *next;
    };
    student **table;
    int Hash(int key)

```

```
{  
    return key%10;  
}
```

public:

```
Student(int total){  
    row = total;  
    table = new student*[row];  
    for(int i=0; i<row; i++)  
        table[i] = NULL;  
}  
void insertByHashing(int, char*);  
void displayTable();  
char* seach(int);
```

```
};
```

```
void Student::insertByHashing(int roll, char *name)
```

```
{  
    int index = Hash(roll);  
    student *stu = new student;  
    stu->rollNum = roll;  
    strcpy(stu->name, name);  
    stu->next = NULL;  
    if(table[index] == NULL)
```

```
        table[index] = stu;
    else{
        student *temp = table[index];
        while(temp->next)
            temp = temp->next;
        temp->next = stu;
    }
}
```

```
void Student::displayTable()
{
    for(int i=0; i<row; i++){
        student *temp = table[i];
        cout<<" "<<i;
        while(temp){
            cout<<" ---> "<<temp->rollNum<<"("<<temp->name<<")";
            temp = temp->next;
        }
        cout<<endl<<endl;
    }
}
```

```
char* Student::seach(int roll)
{
```

```
int index = Hash(roll);
student *temp = table[index];
while(temp){
    if(temp->rollNum == roll)
        return temp->name;
    temp = temp->next;
}
return "Roll Number is Invalid";
}
```

```
int main()
{
    Student s1(10);
    s1.insertByHashing(100, "Rishav");
    s1.insertByHashing(103, "Shahil");
    s1.insertByHashing(105, "Jayant");
    s1.insertByHashing(115, "Bhanu");
    s1.insertByHashing(112, "Rohit");
    s1.insertByHashing(113, "Shivam");
    s1.insertByHashing(109, "Sourav");
    s1.insertByHashing(119, "Ashutosh");
    s1.insertByHashing(148, "Kishan");
    s1.insertByHashing(133, "Amresh");
    s1.displayTable();
}
```

```
int roll = 115;
cout<<endl<<" "<<roll<<" == "<<s1.seach(roll)<<endl;
}
```

Graph Representation With Adjacency Matrix

```
#include<iostream>
using namespace std;

class MatrixGraph
{
    struct Graph
    {
        int vertex;
        int edge;
        int **adj;
    };
public:
    Graph *graph;
    MatrixGraph(int v, int e)
    {
        graph = new Graph;
        graph->vertex = v;
        graph->edge = e;
```

```

graph->adj = new int*[v];
for(int i=0; i<v; i++){
    graph->adj[i] = new int[v];
    for(int j=0; j<v; j++)
        graph->adj[i][j] = 0;
    }
}

void insertEdge(int v1, int v2)
{
    for(int i=0; i<graph->edge; i++){
        graph->adj[v1-1][v2-1] = 1;
        graph->adj[v2-1][v1-1] = 1;
    }
}

};

void viewGraph(MatrixGraph G)
{
    for(int i=0; i<G.graph->vertex; i++){
        for(int j=0; j<G.graph->vertex; j++)
            cout<<G.graph->adj[i][j]<<" ";
        cout<<endl;
    }
}

```

```
}  
  
int main()  
{  
    MatrixGraph G(5,7);  
    G.insertEdge(1,2);  
    G.insertEdge(1,3);  
    G.insertEdge(2,3);  
    G.insertEdge(4,3);  
    G.insertEdge(2,4);  
    G.insertEdge(3,5);  
    G.insertEdge(4,5);  
    viewGraph(G);  
}
```

Graph Linked Representation

```
#include<iostream>  
  
using namespace std;  
  
class LinkedGraph
```



```

{
    struct Node{
        int vertex;
        Node *next;
    };
    struct Graph{
        int vertex, edge;
        Node *table;
    };
    Graph *graph;
public:
    LinkedGraph(int v, int e){
        graph = new Graph;
        graph->vertex = v;
        graph->edge = e;
        graph->table = new Node[v];
        for(int i=0; i<v; i++){
            graph->table[i].vertex = i+1;
            graph->table[i].next = NULL;
        }
    }

    void insertEdge(int v1, int v2)
    {

```

```

Node *n = new Node;
n->vertex = v2;
n->next = NULL;
if(graph->table[v1-1].next == NULL)
    graph->table[v1-1].next = n;
else{
    Node *temp = graph->table+v1-1;
    while(temp->next)
        temp = temp->next;
    temp->next = n;
}
}

```

```

void insertEdge1(int v1, int v2)
{
    Node *n1 = new Node;
    n1->vertex = v2;
    n1->next = NULL;
    if(graph->table[v1-1].next == NULL)
        graph->table[v1-1].next = n1;
    else{
        Node *temp = graph->table+v1-1;
        while(temp->next)
            temp = temp->next;
    }
}

```

```

        temp->next = n1;
    }

    Node *n2 = new Node;
    n2->vertex = v1;
    n2->next = NULL;
    if(graph->table[v2-1].next == NULL)
        graph->table[v2-1].next = n2;
    else{
        Node *temp = graph->table[v2-1].next;
        while(temp->next)
            temp = temp->next;
        temp->next = n2;
    }
}

~LinkedGraph(){
    for(int i=0; i<graph->vertex; i++){
        Node *temp = graph->table[i].next;
        cout<<endl<<"vertex "<<graph->table[i].vertex;
        while(temp){
            cout<<" --> "<<temp->vertex;
            temp = temp->next;
        }
    }
}

```

```

    }
}
};

int main()
{
    LinkedGraph lg(5,7);

    lg.insertEdge1(1,2);
    lg.insertEdge1(1,3);
    lg.insertEdge1(2,3);
    lg.insertEdge1(4,3);
    lg.insertEdge1(2,4);
    lg.insertEdge1(3,5);
    lg.insertEdge1(4,5);
}

```

Depth First Search(logic)

```

void DFS(Graph *G)
{

```

```

Stack *s;
int Status[5]={0},i,x;
s=Create_Stack(G->V);
Push(s,1);
Status[0]=1;
while(s->Top!=-1){
    cout<<Peek(s);
    x=Peek(s)-1;
    Pop(s);
    Status[x]=2;
    for(i=0;i<G->V;i++)
        if(G->Adj[x][i]==1)
            if(Status[i]==0){
                Push(s,i+1);
                Status[i]=1;
            }
    }
}

```

Breadth First Search

```

#include<iostream>
using namespace std;

```

```
int CQ[10], Front=-1,Rare=-1,Size=10;
void insertQ(int value)
{
    if(Rare==Size-1&&Front==0 || Front==Rare+1)
        cout<<"Queue OverFlow";
    else{
        if(Front==-1)
            Rare=Front=0;
        else if(Rare==Size-1)
            Rare=0;
        else
            Rare++;
        CQ[Rare]=value;
    }
}
void deleteQ()
{
    if(Front==-1)
        cout<<"UnderFlow";
    else if(Front==Rare)
        Front=Rare=-1;
    else if(Front==Size-1)
        Front=0;
    else
```

```
        Front++;
    }

class MatrixGraph
{
    struct Graph
    {
        int vertex;
        int edge;
        int **adj;
    };
public:
    Graph *graph;
    MatrixGraph(int v, int e)
    {
        graph = new Graph;
        graph->vertex = v;
        graph->edge = e;
        graph->adj = new int*[v];
        for(int i=0; i<v; i++){
            graph->adj[i] = new int[v];
            for(int j=0; j<v; j++)
                graph->adj[i][j] = 0;
        }
    }
};
```

```

    }
    void insertEdge(int v1, int v2)
    {
        for(int i=0; i<graph->edge; i++){
            graph->adj[v1-1][v2-1] = 1;
            graph->adj[v2-1][v1-1] = 1;
        }
    }
    void BFS();
};

void viewGraph(MatrixGraph G)
{
    for(int i=0; i<G.graph->vertex; i++){
        for(int j=0; j<G.graph->vertex; j++){
            cout<<G.graph->adj[i][j]<<" ";
            cout<<endl;
        }
    }

}

void MatrixGraph::BFS()
{
    int status[graph->vertex] = {0};

```



```

insertQ(1);
status[0] = 1;
while(Front != -1){
    cout<<"v"<<CQ[Front]<<" ";
    status[CQ[Front]-1] = 2;
    for(int i=0; i<graph->vertex; i++)
        if(graph->adj[CQ[Front]-1][i] == 1 && status[i] == 0){
            insertQ(i+1);
            status[i]=1;
        }
    deleteQ();
}
}

```

```

int main()
{
    MatrixGraph G(5,7);
    G.insertEdge(1,2);
    G.insertEdge(1,3);
    G.insertEdge(2,3);
    G.insertEdge(4,3);
    G.insertEdge(2,4);
    G.insertEdge(3,5);
    G.insertEdge(4,5);
}

```

```
G.BFS();
```

```
}
```